

Foundations of 4D/6D Object Capture for Virtual Environments

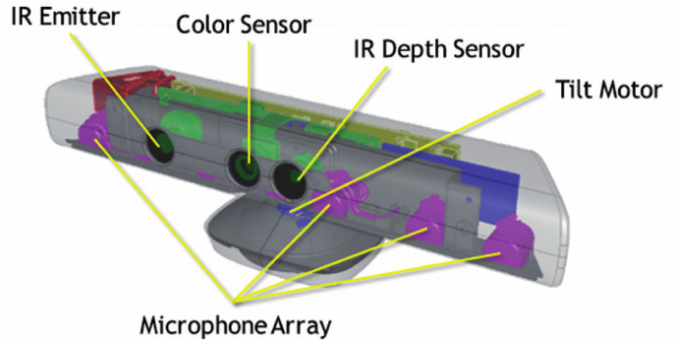


Scene Representations



February 25, 2026

→ **Cameras**



The background of the slide is a virtual scene. On the left, a person wearing a grey and white hoodie and light blue pants stands with one hand on their hip and the other raised. On the right, a white rabbit with a red collar sits. The background is a wall of glowing, star-like patterns. The floor is a dark, flat surface.

Today: Scene Representations

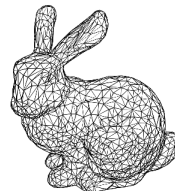
How can we represent 3D objects like the Stanford Bunny?



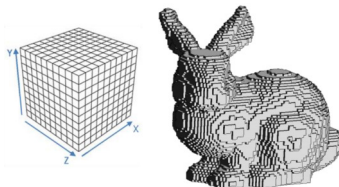
Point Clouds



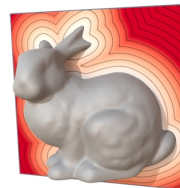
Polygon Mesh



Voxels

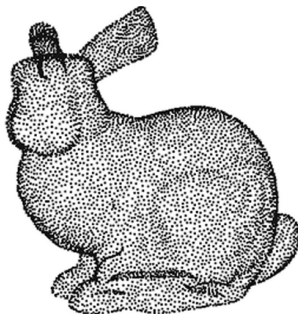


Implicit Surfaces



Discrete set of surface positions in space

$$\mathcal{P} = \{p_1, \dots, p_n \mid p_i \in \mathbb{R}^3\}$$



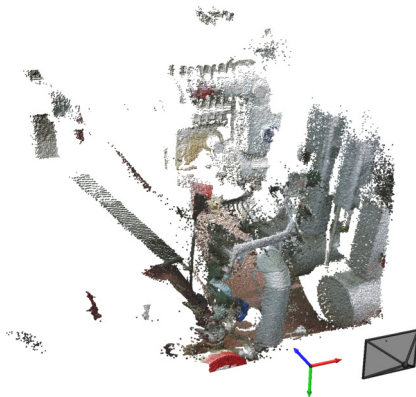
- Simple sampling, just take a subset of the points

Question: How to capture point clouds?

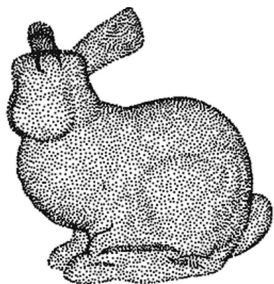
$$\mathcal{P} = \{p_1, \dots, p_n \mid p_i \in \mathbb{R}^3\}$$

Depth images from, e.g., RGB-D cameras can be unprojected to a point cloud

→ Each point can have additional attributes like color



Instead of treating points as "tiny spheres", we can also treat them as small disks



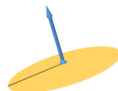
Points

- Position $p_i \in \mathbb{R}^3$

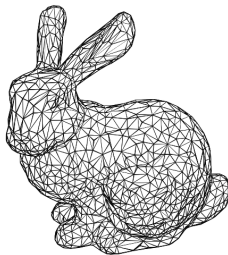


Surfels

- Position $p_i \in \mathbb{R}^3$
- Normal $n_i \in \mathbb{R}^3$
- Radius $r_i \in \mathbb{R}_{>0}$

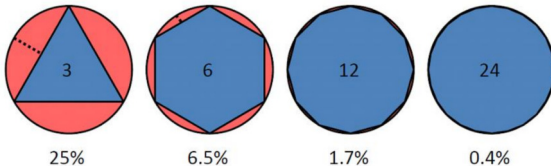


Piecewise linear approximation of the surface



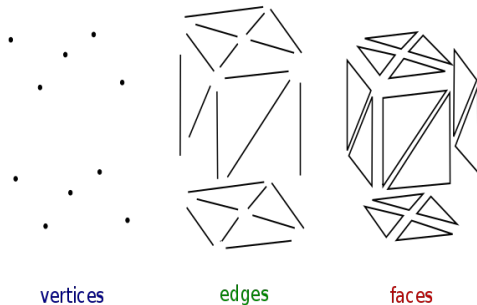
Approximation error is $O(h^2)$:

→ Increasing the resolution by $2\times$ will
roughly reduce error by $4\times$



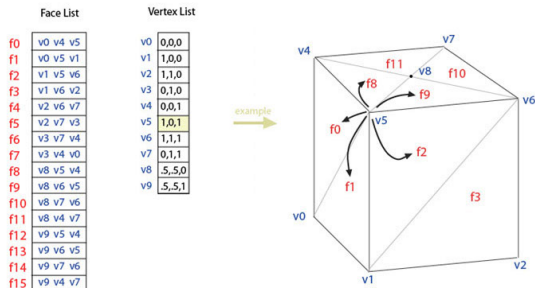
Polygon meshes consist of the following parts:

- ▶ **Vertices:** $\mathcal{V} = \{v_1, \dots, v_n \mid v_i \in \mathbb{R}^3\}$
→ Point cloud
- ▶ **Edges:** Tuple of 2 vertices $E = \{(v_i, v_j) \mid v_i, v_j \in \mathcal{V}\}$
→ Often not directly stored but inferred from faces
- ▶ **Faces:** Tuple of 3, ..., k vertices forming a polygon
→ Usually: $F_{\text{triangles}} = \{(v_i, v_j, v_k) \mid v_i, v_j, v_k \in \mathcal{V}\}$



"Standard" data structure:

- ▶ Vertex list: Point positions, like $|\mathcal{V}| \times 3$ float matrix
- ▶ Face list: Indices of vertices, for triangle meshes: $|\mathcal{F}_{\text{triangles}}| \times 3$ int matrix



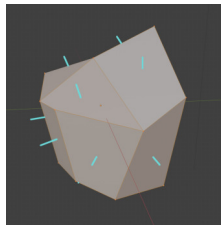
→ Many file formats for storing: OBJ, PLY, STL, ...

Face Normal:

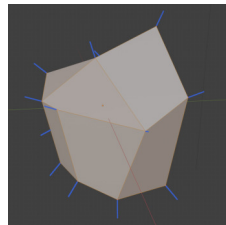
$$\mathbf{n}_{\text{face}}(f_i) = \frac{(\mathbf{v}_{f_{i,1}} - \mathbf{v}_{f_{i,0}}) \times (\mathbf{v}_{f_{i,2}} - \mathbf{v}_{f_{i,0}})}{\|(\mathbf{v}_{f_{i,1}} - \mathbf{v}_{f_{i,0}}) \times (\mathbf{v}_{f_{i,2}} - \mathbf{v}_{f_{i,0}})\|}$$

Vertex Normal:

→ (Weighted) average of adjacent face normals

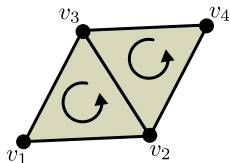


Face Normal



Vertex Normal

Example:



$$f_0 : (1, 2, 3)$$

$$f_1 : (3, 2, 4)$$

$$\mathbf{n}_{\text{face}}(f_0) = \frac{(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)}{\|(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)\|}$$

$$\mathbf{n}_{\text{face}}(f_1) = \frac{(\mathbf{v}_2 - \mathbf{v}_3) \times (\mathbf{v}_4 - \mathbf{v}_3)}{\|(\mathbf{v}_2 - \mathbf{v}_3) \times (\mathbf{v}_4 - \mathbf{v}_3)\|}$$

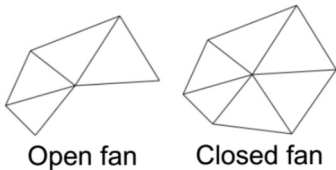
Question: Are all meshes equally "good"? → No!

Definition (Manifold Mesh)

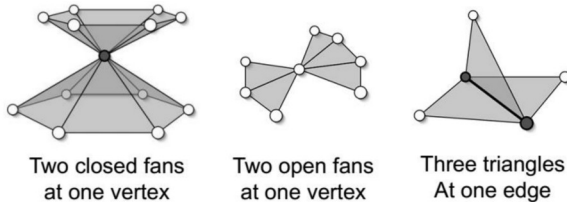
A polygon mesh is called *manifold* if

1. (Self Intersection) No two faces of the mesh intersect
2. (Edge Manifold) Each edge is adjacent to 1 or 2 faces
3. (Vertex Manifold) Faces around each vertex form a single open or closed fan

Manifold



Non-Manifold

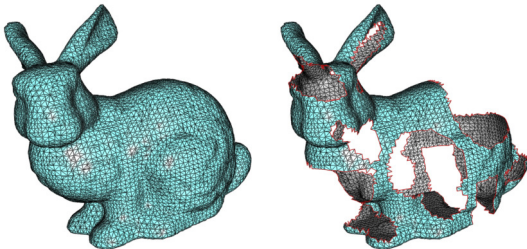


Question: Are all meshes equally "good"? → No!

Definition (Watertight Mesh)

A polygon mesh is called *watertight* if

1. it is a manifold
2. each edge is adjacent to exactly 2 faces



→ Watertight meshes do not have boundary edges

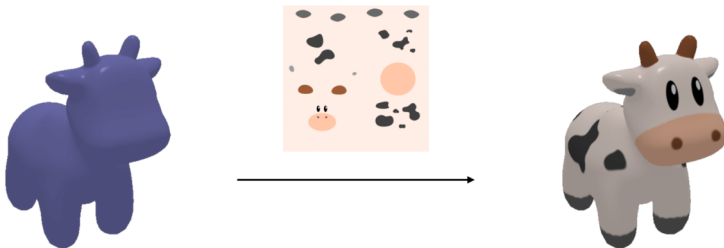
Like for point clouds, we can assign colors to each vertex v_i

→ Limited to mesh resolution

Solution: Map texture onto the mesh

- ▶ Image texture can be accessed by $(u, v) \in [0, 1]^2$
- ▶ Each vertex index in face list is associated with uv coordinate
- ▶ Interpolate uv coordinates inside face

→ Independent of mesh resolution



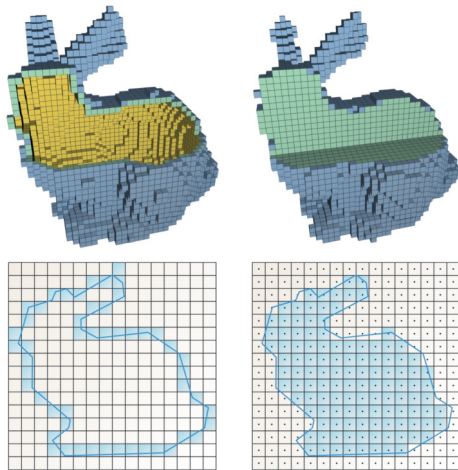
Simple voxel grid with occupancy values

$$\mathcal{G} = (o_{111}, \dots, o_{mnk}) \in \mathbb{R}^{m \times n \times k}$$

→ Common case: Cube where $m = n = k$

Can create boundary or solid geometry

- ▶ Boundary: Check if voxel intersects with object
- ▶ Solid: Check if voxel lies inside of object



Definition (Level surface)

Let $g: \mathbb{R}^3 \rightarrow \mathbb{R}$ be a continuous function and $l \in \mathbb{R}$. The set

$$\mathcal{I} = \{\mathbf{p} \in \mathbb{R}^3 \mid g(\mathbf{p}) = l\}$$

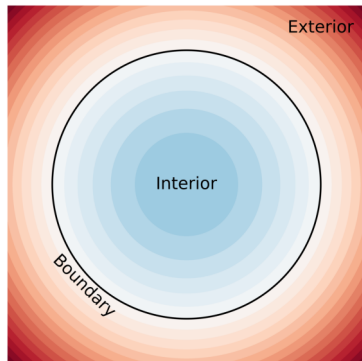
is called the *level surface* or *isosurface* at level l .

→ Most common case: $l = 0$

Implicit surfaces decompose the volume into three parts:

- ▶ $\{\mathbf{p} \in \mathbb{R}^3 \mid g(\mathbf{p}) < 0\}$: Interior
- ▶ $\{\mathbf{p} \in \mathbb{R}^3 \mid g(\mathbf{p}) = 0\}$: Boundary
- ▶ $\{\mathbf{p} \in \mathbb{R}^3 \mid g(\mathbf{p}) > 0\}$: Exterior

Note: g does not have to be a distance, it can be anything, e.g. step function.



2D Example:

$$g(x, y) = x^2 + y^2 - r^2$$

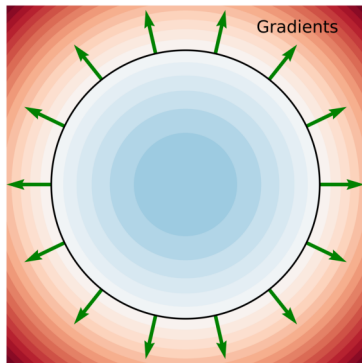
An important quantity of a implicit surface are its *gradients*:

$$\nabla g(\mathbf{p}) = \left(\frac{\partial g}{\partial x}(\mathbf{p}), \frac{\partial g}{\partial y}(\mathbf{p}), \frac{\partial g}{\partial z}(\mathbf{p}) \right)^T$$

Note: At the isosurface $\mathcal{I}$, the normals $\mathbf{n}$ point into the same direction as the gradients $\nabla g(\mathbf{p})$ but may have different lengths:

$$\mathbf{n} \propto \nabla g(\mathbf{p})$$

→ With flipped signs ("positive inside, negative outside"), the gradient would point in the *opposite direction*!



2D Example:

$$g(x, y) = x^2 + y^2 - r^2$$

$$\nabla g(x, y) = (2x, 2y)^T$$

Definition (Signed Distance Function (SDF))

Given an object defined by its interior Ω , boundary $\partial\Omega$, and exterior $\mathbb{R}^3 \setminus \{\Omega \cup \partial\Omega\}$. Then, its *signed distance function* is

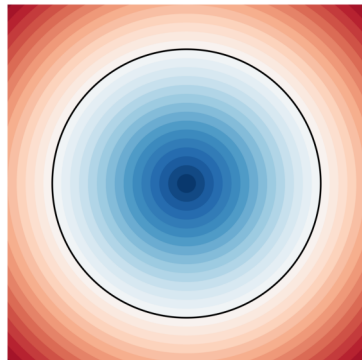
$$\text{sdf}(\mathbf{p}) = \begin{cases} -d(\mathbf{p}, \partial\Omega) & \text{if } \mathbf{p} \in \Omega \\ 0 & \text{if } \mathbf{p} \in \partial\Omega \\ d(\mathbf{p}, \partial\Omega) & \text{if } \mathbf{p} \in \mathbb{R}^3 \setminus \{\Omega \cup \partial\Omega\} \end{cases}$$

where $d(\mathbf{p}, \partial\Omega) = \inf_{\mathbf{q} \in \partial\Omega} d(\mathbf{p}, \mathbf{q})$ is the closest distance of $\mathbf{p}$ to $\partial\Omega$ according to a metric d .

Typically, we only use the Euclidean distance $d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|$

→ Signed distance function also fulfills *Eikonal equation*:

$$\|\nabla \text{sdf}(\mathbf{p})\| = 1$$



2D Example:

$$g(x, y) = \|(x, y)^T\| - r$$

Applications:

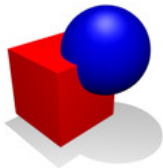
- ▶ Reconstruction methods
 - Data Fusion, more on that later
- ▶ Game engines like Unreal Engine
 - Dynamic Ambient Occlusion
 - Shadow Computation
 - Particle Collision



Implicit surfaces can be "easily" combined via *boolean operations*:

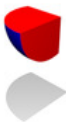
Union

$$g_1 \cup g_2 = \min(g_1, g_2)$$



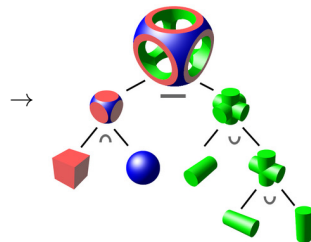
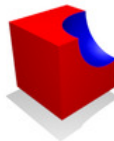
Intersection

$$g_1 \cap g_2 = \max(g_1, g_2)$$



Difference

$$g_1 \setminus g_2 = \max(g_1, -g_2)$$



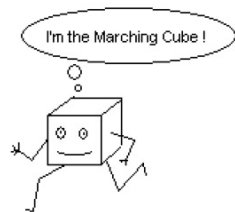
Note: Boolean operations depend on the sign for interior and exterior

Problem: How to get an explicit representation of the isosurface?

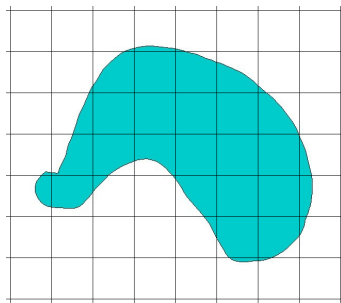
Solution: [Marching Cubes](#) algorithm¹

- ▶ Approximates the surface by a triangle mesh
- ▶ Surface is found by linear interpolation along cell edges
- ▶ Uses gradients as the normals of the isosurface
- ▶ Efficient computation by means of lookup tables

→ THE standard geometry-based isosurface extraction algorithm



¹W. E. Lorensen and H. E. Cline. "Marching cubes: A high resolution 3D surface construction algorithm". In: *Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. vol. 21. 4. ACM. 1987.



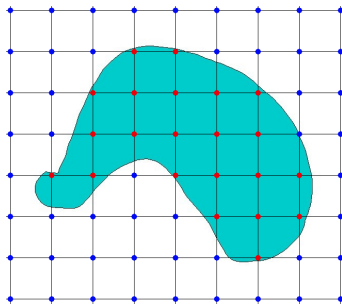
Task: Compute a mesh from the above 2D object → Generalization to 3D is straightforward

Given: Subdivided space:

► Sampled scalar values $g(i, j, k) \dots$

► ... at each grid point $(i, j, k)^T \in \mathbb{Z}^3$

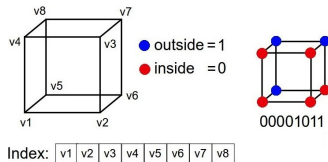
g is oftentimes a signed distance function sdf
→ In general, g can be **any continuous function!**

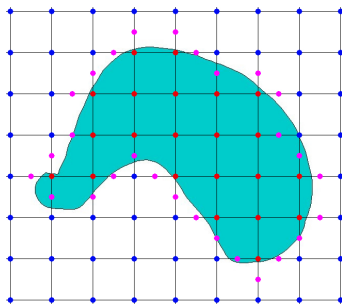


Step 1: For each grid point $(i, j, k)^T \in \mathbb{Z}^3$ decide whether it is **outside** or **inside** the object

$$g(i, j, k) > 0 \quad , \quad g(i, j, k) \leq 0$$

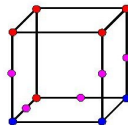
and calculate an index from $i + [0, 1], j + [0, 1], k + [0, 1]$

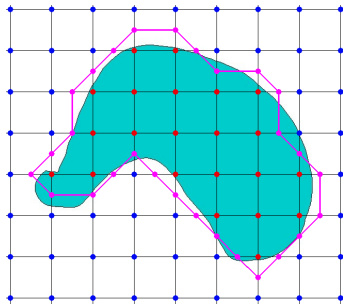




Step 2 (Simple Version): Compute mesh vertices as **midpoints** by using indices from a lookup table

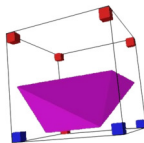
→ Lookup table stores the locations of the corners having different labels based on the index





Step 3: Connect the mesh vertices through **lines (faces)** by using indices from a lookup table

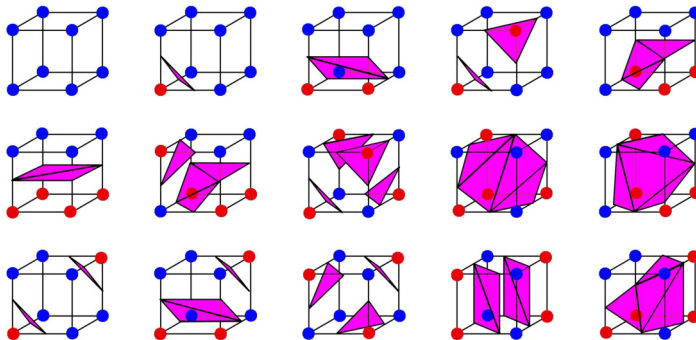
→ Lookup table stores the triangulation of the mesh vertices



Step 3: Connect the mesh vertices through lines (faces) by using indices from a lookup table

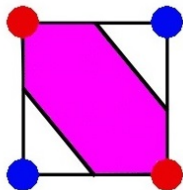
→ Lookup table stores the triangulation of the mesh vertices

→ All 256 cases can be derived from 15 base cases due to symmetries

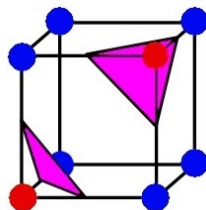
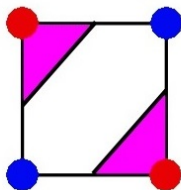


Problem: Ambiguous cases

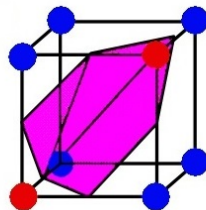
- ▶ Adjacent vertices: different states
- ▶ Diagonal vertices: same state



or

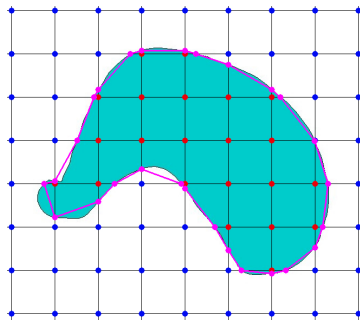
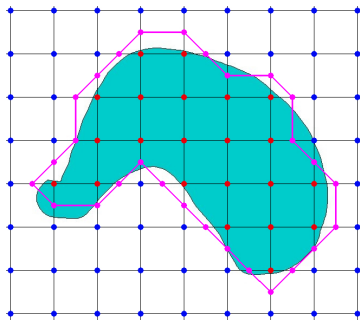


or



Solution: To avoid holes in the reconstruction, decide for one solution **consistently** for all possible cases!

Problem: Choosing midpoints leads to "blocky" approximation of the actual surface



Step 2 (Standard Version): Scalar values $g(i, j, k)^T$ indicate some sort of "distance" to the surface
→ Use **linear interpolation** to compute mesh vertices

$$\mathbf{v}_{\text{mesh}} = \mathbf{v}_1 + \frac{0 - g(\mathbf{v}_1)}{g(\mathbf{v}_2) - g(\mathbf{v}_1)} \cdot (\mathbf{v}_2 - \mathbf{v}_1)$$

Optional Step: Calculate the normal at each triangle vertex

- Calculate (approximate) gradient of the continuous function g at each cube vertex (central differences):

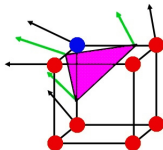
$$\nabla g(i, j, k) \approx \hat{\nabla} g(i, j, k) = \begin{pmatrix} g(i+1, j, k) - g(i-1, j, k) \\ g(i, j+1, k) - g(i, j-1, k) \\ g(i, j, k+1) - g(i, j, k-1) \end{pmatrix}$$

- Compute normal from (approximate) gradient:

$$\mathbf{n} = \frac{\hat{\nabla} g(i, j, k)}{\|\hat{\nabla} g(i, j, k)\|}$$

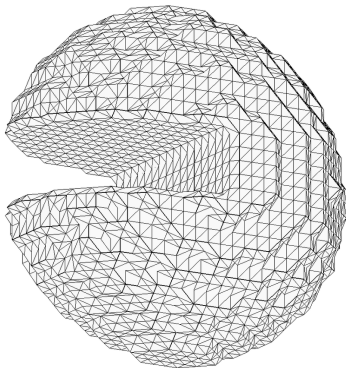
Note that normalization is needed since the gradient is not exact

- Use linear interpolation to compute the polygon vertex normal (of the isosurface)



Problem: Extracted mesh may not be smooth and may have close to degenerate triangles

→ Depends on smoothness of implicit surface and discrete grid resolution



Idea: Smooth the mesh by reducing the curvature

Definition (Laplace Operator)

The *Laplace Operator* or *Laplacian* of a twice continuously differentiable function f is defined as

$$\Delta f := \operatorname{div} \nabla f = \sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2}$$

With the Laplace operator, we have a powerful tool to measure curvature:

Theorem (Mean Curvature Normal)

The *Mean Curvature* κ_M of a point p can be computed using

$$\Delta_M p = -2 \kappa_M n$$

where $\Delta_M f = \operatorname{div}_M \nabla_M f$ is called the *Laplace-Beltrami Operator* which is a generalization of the Laplacian on surfaces (manifolds).

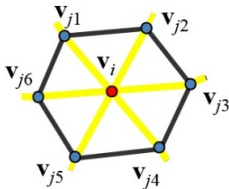
Question: How can we compute the Laplacian on triangle meshes?

$$\Delta_M \mathbf{v}_i \approx \mathbf{L}(\mathbf{v}_i) = \frac{\sum_{\mathbf{v}_j \in \mathcal{N}(\mathbf{v}_i)} w_{ij} (\mathbf{v}_j - \mathbf{v}_i)}{\sum_{\mathbf{v}_j \in \mathcal{N}(\mathbf{v}_i)} w_{ij}}$$

Uniform Weights

$$w_{ij} = 1$$

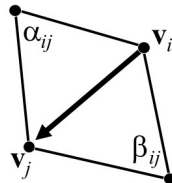
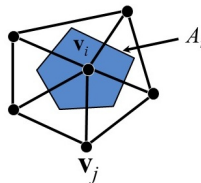
→ Assumes regular triangulation



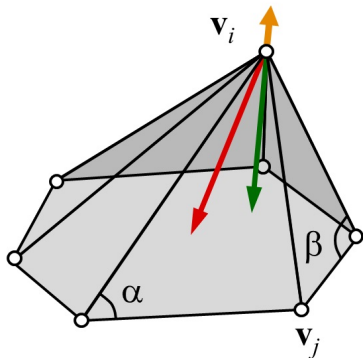
Cotangent Weights

$$w_{ij} = \frac{1}{2} (\cot \alpha_{ij} + \cot \beta_{ij})$$

→ Accounts for shape/area of triangles



Observation: When the edge lengths are equal, the uniform and the cotangent Laplacians coincide



Uniform Laplacian $L_u(v_i)$
Cotangent Laplacian $L_c(v_i)$
Mean curvature normal

For nearly equal edge lengths
Uniform $\approx$ Cotangent

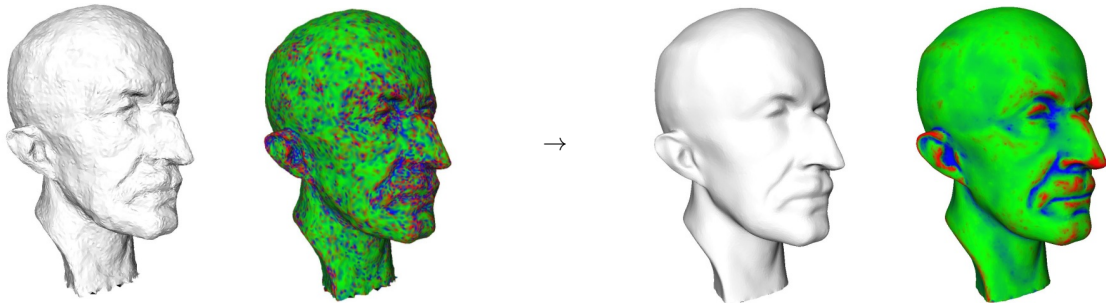
Note: As a trick, the uniform Laplacian can be intentionally used to make the triangle shapes more regular!

Idea:

- ▶ Laplacian points into the **opposite** direction as the curvature-scaled normal
- ▶ Iteratively apply Laplacian on mesh

$$\mathbf{v}_i^{(t+1)} = (\mathbf{I} + \lambda \cdot \mathbf{L}) \mathbf{v}_i^{(t)}$$

→ Laplacian Smoothing



Question: What if we only need a single "view" of the implicit surface?

Idea: Cast rays and find positions where the rays hit the isosurface

- Rays are generated as

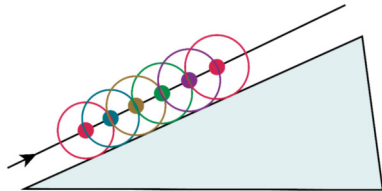
$$\mathbf{r}(t) = \mathbf{r}_0 + t \cdot \mathbf{r}_d$$

with ray origin $\mathbf{r}_0 \in \mathbb{R}^3$, ray direction $\mathbf{r}_d \in \mathbb{R}^3$ with $\|\mathbf{r}_d\| = 1$,
and $t \in \mathbb{R}_{\geq 0}$

- Use equidistant point sampling

$$t_{i+1} = t_i + \Delta t \quad t_0 = 0, \text{ step size } \Delta t$$

- When detecting that isosurface has been crossed, then repeat with smaller denser sampling on that interval



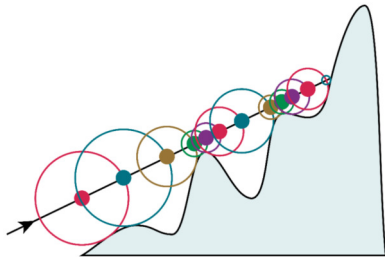
Problem: Ray Marching is quite slow

→ Can be accelerated if implicit surface is a signed distance function²

- ▶ Since we know that SDF describes the *minimum distance* to the surface along some direction, we can safely traverse by that amount
- ▶ Use SDF-based point sampling

$$t_{i+1} = t_i + \text{sdf}(\mathbf{r}(t_i)) \quad t_0 = 0$$

- ▶ Becomes increasingly slower the closer to the surface



²J. C. Hart. "Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces". In: *The Visual Computer* 12:10 (1996).

Next Lecture: Reconstruction Methods

→ Reconstruction Methods



- [1] J. C. Hart. "Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces". In: *The Visual Computer* 12.10 (1996).
- [2] W. E. Lorensen and H. E. Cline. "Marching cubes: A high resolution 3D surface construction algorithm". In: *Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. Vol. 21. 4. ACM. 1987.